

My First RAG Application

Self-Learning Guide: Groq + ChromaDB + Local Embeddings

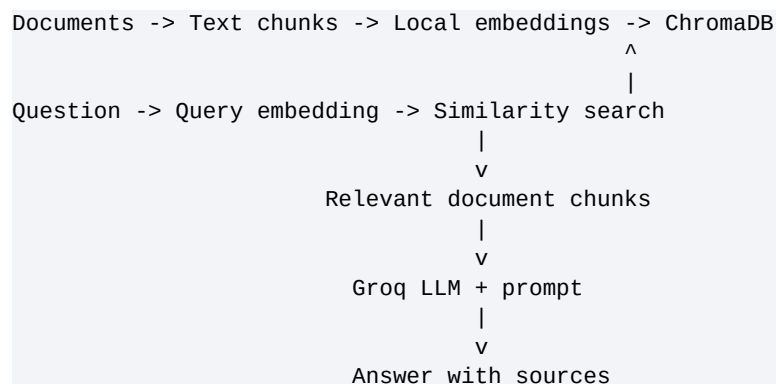
Beginner Edition | 2026

A beginner-friendly, self-learning project that teaches how to build a complete Retrieval-Augmented Generation (RAG) application.

What you will build

You will build an application that:

1. Reads TXT, PDF and DOCX files.
2. Splits long text into smaller chunks.
3. Converts chunks into numerical embeddings on your own computer.
4. Stores the embeddings in a local ChromaDB vector database.
5. Finds document chunks related to a user's question.
6. Sends only the retrieved evidence and question to a Groq-hosted language model.
7. Displays the generated answer together with the retrieved sources.



Why this project is simple

- No LangChain or LlamaIndex is used. You can see each RAG step directly.
- ChromaDB runs locally and saves data in the `chroma\_db` folder.
- Embeddings run locally with `all-MiniLM-L6-v2`, so no embedding API bill is required.
- Groq is used only for response generation.
- A sample employee handbook is included.
- The project has a command line, web screen and REST API.

Groq may provide free developer access subject to current account limits and policies. It is not guaranteed to remain unlimited or permanently free. Never place your API key in GitHub.

Lesson 1 - Understand RAG

What is an LLM?

A Large Language Model predicts and generates text. It knows general information from its training, but it does not automatically know your private documents.

What is a hallucination?

A hallucination is an answer that sounds confident but is unsupported or incorrect.

What is RAG?

RAG means Retrieval-Augmented Generation.

- **Retrieval:** Find relevant text from your documents.
- **Augmentation:** Add that text to the model prompt.
- **Generation:** Ask the model to answer using the retrieved text.

RAG is not model training or fine-tuning. It supplies external knowledge at question time.

Key terms

Term	Simple meaning
Document	A PDF, Word or text file
Chunk	A small portion of a document
Embedding	A list of numbers representing the meaning of text
Vector database	A database that stores embeddings and finds similar vectors
Similarity search	Finding chunks whose meaning is close to the question
Context	Retrieved text supplied to the LLM
Prompt	Instructions and input sent to the LLM
API key	A secret credential used to call Groq

Lesson 2 - Install the software

2.1 Install Python

Install Python 3.11 or newer from the official Python website.

Verify it:

```
python --version
```

On macOS or Linux, try:

```
python3 --version
```

2.2 Install Visual Studio Code

Install VS Code and open this project folder using File -> Open Folder.

2.3 Optional: install Git

Verify Git:

```
git --version
```

Lesson 3 - Download and prepare the project

Option A: download ZIP

Download this repository as a ZIP, extract it, and open the extracted folder in VS Code.

Option B: clone from GitHub

```
git clone YOUR_REPOSITORY_URL  
cd groq-rag-starter
```

Create a virtual environment

Windows PowerShell

```
python -m venv .venv  
.venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, run this once in the same window:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.venv\Scripts\Activate.ps1
```

Windows Command Prompt

```
python -m venv .venv  
.venv\Scripts\activate.bat
```

macOS or Linux

```
python3 -m venv .venv  
source .venv/bin/activate
```

When activated, the terminal normally begins with (.venv).

Install dependencies

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

The first installation can take several minutes because the local embedding library includes machine-learning dependencies.

Lesson 4 - Get and protect a Groq API key

1. Create or sign in to a GroqCloud account.
2. Open the API Keys section.
3. Create a new API key.

4. Copy `.env.example` to a new file called `.env`.

5. Replace the placeholder value.

Windows

```
copy .env.example .env
```

macOS or Linux

```
cp .env.example .env
```

Your `.env` should look like this:

```
GROQ_API_KEY=gsk_your_real_key_here
GROQ_MODEL=llama-3.1-8b-instant
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
CHROMA_PATH=chroma_db
COLLECTION_NAME=student_rag_documents
TOP_K=4
```

Security rules

- Do not send your key to another person.
- Do not show it in screenshots.
- Do not commit `.env` to GitHub.
- If a key is exposed, delete/revoke it and create another one.
- `.gitignore` already excludes `.env`.

Lesson 5 - Inspect the project

```
app/
  config.py      Reads environment settings
  documents.py   Reads and chunks TXT, PDF and DOCX files
  vector_store.py Creates embeddings and uses ChromaDB
  rag.py         Retrieves evidence and calls Groq

data/
  employee_handbook.txt Sample knowledge document

ingest.py       Loads documents into the vector database
cli.py          Terminal chatbot
streamlit_app.py Web interface
api.py          FastAPI REST service
tests/         Beginner automated tests
```

Read the files in this order:

1. `app/documents.py`

2. `app/vector_store.py`

3. `app/rag.py`

4. `ingest.py`

5. `streamlit_app.py`

Lesson 6 - Build the vector database

Run:

```
python ingest.py --reset
```

On the first run, the embedding model is downloaded. Internet access is required once. Later runs can use the cached model.

Expected output is similar to:

```
Success: stored 4 chunks in ChromaDB.
```

The exact chunk count can change if the sample text or chunk settings change.

What happened?

1. The code found files inside data/.
2. It extracted text.
3. It split text into overlapping chunks.
4. The local sentence-transformer converted each chunk into a vector.
5. ChromaDB stored the vector, original text and source metadata.

Why overlap chunks?

Without overlap, a sentence near a boundary could be separated from related information. Overlap keeps some neighboring text in both chunks.

Lesson 7 - Run the command-line assistant

```
python cli.py
```

Try these questions:

```
How many annual leave days do employees receive?  
How many unused leave days can be carried forward?  
When is a medical certificate required?  
What security rules apply to remote workers?  
What is the health insurance amount?
```

The final question is not answered by the sample document. A well-grounded result should say the information could not be found.

Type exit to stop.

Lesson 8 - Run the web application

```
streamlit run streamlit_app.py
```

A browser should open automatically. If not, open the local URL shown in the terminal, usually:

```
http://localhost:8501
```

Use the sidebar button to rebuild the vector database after changing documents.

Lesson 9 - Run the REST API

```
uvicorn api:app --reload --port 8000
```

Open the interactive API page:

```
http://127.0.0.1:8000/docs
```

Use POST /ask with:

```
{
  "question": "How many annual leave days are available?",
  "top_k": 4
}
```

Health check:

```
http://127.0.0.1:8000/health
```

Lesson 10 - Add your own documents

1. Stop the application.
2. Add .txt, .pdf or .docx files to data/.
3. Rebuild the vector database:

```
python ingest.py --reset
```
4. Restart Streamlit or the API.
5. Ask questions whose answers are present in the documents.

Important PDF limitation

pypdf extracts digital text. A scanned PDF is mainly an image and normally requires OCR before ingestion.

Data privacy warning

Although embeddings are created locally, retrieved document text is sent to Groq in the generation prompt. Do not use confidential, regulated or personal data unless you have authorization and have reviewed the provider's current data-handling terms.

Lesson 11 - Follow one question through the code

Suppose the user asks:

```
How much annual leave do employees receive?
```

Step A: create the query embedding

app/vector_store.py converts the question into a normalized vector.

Step B: search ChromaDB

ChromaDB compares the query vector with stored document vectors and returns the nearest chunks.

Step C: construct context

app/rag.py labels each retrieved chunk:

```
[SOURCE 1: employee_handbook.txt, page 1]
Every full-time employee receives 24 days...
```

Step D: call Groq

The prompt tells the model to:

- answer only from context;
- say when information is missing;
- cite source labels;
- avoid invented facts.

Step E: display evidence

The UI shows the answer and retrieved chunks so the user can inspect the evidence.

Lesson 12 - Run automated tests

```
pytest -q
```

The included tests check chunking and source-context formatting. They do not call Groq, so they do not consume API usage.

Lesson 13 - Student exercises

Exercise 1: Change sample knowledge

Add a new section to employee_handbook.txt, rebuild the database and ask a question about it.

Exercise 2: Adjust retrieval count

Change TOP_K in .env from 4 to 2. Observe how fewer chunks are supplied.

Exercise 3: Observe semantic retrieval

Add this sentence:


```
Employees receive reimbursement for approved professional learning courses.
```

Ask:

```
Does the company pay for staff education?
```

The words differ, but semantic search may still retrieve the relevant chunk.

Exercise 4: Improve metadata

Add a department field to stored metadata and display it in the UI.

Exercise 5: Add a file upload screen

Extend Streamlit so a user can upload a supported document and rebuild the database.

Exercise 6: Add chat history

Use `st.session_state` to preserve previous questions and answers.

Exercise 7: Evaluate the RAG system

Create ten questions with expected answers and sources. Record whether retrieval and generation are correct.

Lesson 14 - Troubleshooting

`GROQ\_API\_KEY is missing`

Confirm that:

- the file is named exactly `.env``;
- it is in the project root;
- the key is after ``GROQ_API_KEY=``;
- the terminal/application was restarted after editing.

`The vector database is empty`

Run:

```
python ingest.py --reset
```

`No TXT, PDF or DOCX files found`

Add a supported file to `data/` and run ingestion again.

Python command not found

Reinstall Python and select Add Python to PATH on Windows.

PowerShell activation is blocked

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.venv\Scripts\Activate.ps1
```

Package installation fails

```
python -m pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
```

Model not found or retired

Groq model availability can change. Check Groq's current supported-model page, then update GROQ_MODEL in .env.

Poor retrieval

Try:

- clearer documents;
- smaller or larger chunks;
- increasing `TOP\_K`;
- removing headers/footers repeated on every PDF page;
- using a stronger embedding model;
- adding reranking later.

Hallucinated answer

- Make the system prompt stricter.
- Reduce generation temperature.
- inspect retrieved evidence;
- add an evidence-quality threshold;
- improve document chunking;
- evaluate with known questions.

Lesson 15 - How to publish this repository

Create a new GitHub repository

Create an empty repository named, for example:

```
groq-rag-starter
```

Do not ask GitHub to add a README because this project already has one.

Push from the project folder

```
git init
git add .
git commit -m "Initial beginner Groq RAG project"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Before pushing, verify that .env is excluded:

```
git status
```

.env must not appear in the files to be committed.

Architecture decisions

Why local embeddings?

Groq is used for language-model inference, while Sentence Transformers creates embeddings locally. This makes the learning project inexpensive and clearly separates retrieval from generation.

Why ChromaDB?

It is easy to install, can persist locally without a separate server, stores text and metadata, and supports nearest-neighbor vector queries.

Why no LangChain?

Frameworks are useful later, but a beginner should first understand document loading, chunking, embeddings, retrieval, prompt construction and generation directly.

Why `llama-3.1-8b-instant`?

It is a small, fast Groq production model at the time this guide was prepared. Model availability and free-tier limits can change, so the model is configured through `.env` rather than hard-coded everywhere.

Production checklist

This repository is for learning. Before using it professionally, consider:

- user authentication and authorization;
- document-level access controls;
- malware scanning and file-size limits;
- OCR and better document parsing;
- encryption and secrets management;
- prompt-injection defenses;
- retrieval score thresholds;
- hybrid keyword and vector search;
- reranking;
- API rate limiting;
- logging without leaking sensitive text;
- automated RAG evaluation;
- cost and token monitoring;
- provider data-retention review;
- server-backed vector database and backups.

Completion checklist

The student has completed the course when they can:

- ☐ Explain RAG in their own words.
- ☐ Protect an API key using ``.env``.
- ☐ Explain document chunking and overlap.
- ☐ Explain embeddings and vector similarity.
- ☐ Ingest their own document into ChromaDB.
- ☐ Run the command-line application.
- ☐ Run the Streamlit application.
- ☐ Test the FastAPI endpoint.
- ☐ Identify a hallucination or unsupported answer.
- ☐ Show which retrieved evidence supports an answer.
- ☐ Push the project to GitHub without exposing the API key.

License

MIT - suitable for learning, modification and sharing.